Cahier des charges : détection statistique des anomalies

Phase 2 du projet « Gestion intelligente des logs »

Destinataire : membre de l'équipe en charge de la détection · Rédigé par l'administrateur du groupe
Version 1.0 · 06/11/2026 · Prérequis : phase 1 terminée (RHEL → MongoDB validé)

1. Contexte et objectif

La phase 1 est terminée : les logs réels de RHEL sont reçus, normalisés en `LogEvent` et enregistrés dans MongoDB (base `gestion_logs`, collection `logs`). Cette phase ajoute l'**analyse statistique** de ces logs pour repérer automatiquement des **comportements inhabituels**.

Objectif : produire un service qui, à intervalle régulier, compare l'activité récente à l'activité habituelle de chaque machine, détecte les écarts significatifs, leur attribue un **score** et un **niveau**, puis les enregistre sous forme d'objets `Anomaly` dans MongoDB pour que l'API REST et le frontend React (phases 3 et 4) puissent les afficher.

> **Important**
> Il ne s'agit **pas** des règles textuelles existantes (`DetectionRule` / `RuleDetectionService`, qui cherchent un mot comme `ERROR` dans un message). Elles restent telles quelles. Ici, on détecte des écarts **statistiques** sur des volumes et des proportions.

2. Périmètre

Inclus (obligatoire)	Hors périmètre de cette phase
• Détection d'un **pic de fréquence** de logs par fenêtre de temps • Détection d'une **concentration anormale de logs de sévérité élevée** • Score d'anomalie et niveau (MEDIUM / HIGH / CRITICAL) • Modèle `Anomaly`, stockage MongoDB, repository • Service d'analyse périodique, indépendant du récepteur • Tests unitaires et test réel avec RHEL	• API REST, authentification, frontend React • Apprentissage automatique (machine learning) • Modification du format des logs ou du parseur • Envoi d'alertes (e-mail, SMS) • Saisonnalité (comparaison heure par heure) : amélioration possible, voir section 5.7

3. Existant sur lequel s'appuyer

Flux actuel : `SyslogTcpReceiver` → `LogNormalizer` → `LogEvent` → `LogIngestionService` → `MongoLogRepository` → MongoDB. La détection se branche **après** ce flux, en lecture seule sur les logs.

Classe existante	Ce que vous utiliserez
`model.LogEvent`	Champs : `id`, `eventTimestamp`, `receivedTimestamp`, `hostname`, `service`, `severity`, `category`, `message`, `sourceAddress`, `username` (non rempli pour l'instant)
`repository.LogRepository`	`save`, `findAll`, `findByHostname`, `findBySeverity`, `findByPeriod(Instant from, Instant to)` (borne `from` incluse, `to` exclue, tri du plus récent)
`repository.MongoLogRepository`	Implémentation MongoDB ; sert de modèle de code pour le repository des anomalies

Classe existante	Ce que vous utiliserez
`repository.MongoConnection`	Connexion partagée ; ajoutez-y un accès à la collection `anomalies` (méthode supplémentaire)
`functional.AnomalyScorer`	Interface `double calculateScore(LogEvent)` : travaille sur **un** événement. Ne la modifiez pas ; la détection par fenêtre n'en dépend pas

Valeurs de `severity` produites par le normaliseur : `EMERGENCY`, `ALERT`, `CRITICAL`, `ERROR`, `WARNING`, `NOTICE`, `INFO`, `DEBUG`. Valeurs de `category` : au minimum `SYSLOG` et `UNPARSED`.

> **Règles à respecter**
> • N'altérer aucune signature existante (`LogRepository`, `LogIngestionService`, `LogNormalizer`…).
> • Ne rien ajouter qui ralentisse le récepteur : l'analyse tourne dans un **thread séparé**.
> • Utiliser `receivedTimestamp` (toujours renseigné) pour les fenêtres, pas `eventTimestamp` (absent pour les logs `UNPARSED`, et année supposée pour le RFC 3164).
> • Tout changement dans `pom.xml`, `LogRepository` ou `SyslogTcpReceiver` se fait **avec l'accord de l'administrateur**.

4. Exigences fonctionnelles

ID	Exigence	Priorité
EF1	Découper le temps en **fenêtres** de durée fixe (défaut : 60 s) alignées sur des bornes régulières	Obligatoire
EF2	Calculer, par fenêtre et **par hostname** (et globalement, hostname `ALL`), le nombre de logs	Obligatoire
EF3	Calculer, par fenêtre et par hostname, la **proportion de logs de sévérité élevée** (`EMERGENCY`, `ALERT`, `CRITICAL`, `ERROR`)	Obligatoire
EF4	Établir une **référence (baseline)** : moyenne et écart-type sur les N fenêtres précédentes	Obligatoire
EF5	Détecter un **pic de fréquence** : `FREQUENCY_SPIKE`	Obligatoire
EF6	Détecter une **concentration de sévérité élevée** : `HIGH_SEVERITY_CONCENTRATION`	Obligatoire
EF7	Calculer un **score** entre 0 et 1 et un **niveau** pour chaque anomalie	Obligatoire
EF8	Enregistrer chaque anomalie dans MongoDB (collection `anomalies`) **sans doublon** en cas de ré-analyse	Obligatoire
EF9	Lancer l'analyse automatiquement toutes les 60 s, et aussi en exécution manuelle sur les logs existants	Obligatoire
EF10	Paramètres modifiables sans toucher à l'algorithme (classe de configuration)	Obligatoire
EF11	Détecter une **chute brutale** de volume (`VOLUME_DROP`, source possiblement en panne)	Optionnel
EF12	Méthode **robuste** (médiane et MAD) en alternative à la moyenne et l'écart-type	Optionnel

5. Spécification statistique

5.1 Fenêtres et séries de mesures

Pour une durée de fenêtre `W`, la fenêtre cible est la dernière fenêtre **complète** `[fin − W ; fin[`, où `fin` est l'instant courant arrondi à un multiple de `W`. Pour chaque hostname on construit, sur les `N` fenêtres qui la précèdent, deux séries :

- **M1 (volume)** : nombre de logs reçus dans chaque fenêtre ;

- **M2 (sévérité)** : proportion `r = logs de sévérité élevée / logs totaux`, calculée seulement pour les fenêtres contenant au moins un log.

> **Important**
> Une fenêtre **sans aucun log compte pour 0** dans la série M1. Si on ne l'inclut pas, la moyenne est surestimée et les pics passent inaperçus. Il faut donc générer explicitement toutes les fenêtres vides.

5.2 Référence (baseline)

Sur la série M1 des `N` fenêtres précédentes (la fenêtre cible est **exclue**) :

```
moyenne  μ = (x1 + x2 + ... + xN) / N
variance s² = Σ (xi − μ)² / (N − 1)          (écart-type d'échantillon)
écart-type s = racine(s²)
écart-type effectif  s_eff = max(s ; s_min)     s_min = 1,0 pour M1 ; 0,05 pour M2
```

`s_min` évite une division par zéro (ou un score explosif) quand l'historique est parfaitement plat.

5.3 Score d'anomalie et niveaux

```
z = (x − μ) / s_eff                 x = valeur de la fenêtre cible
score = min(1 ; max(0 ; z / z_max))    z_max = 6   (paramétrable)
anomalie  si  z >= z_seuil             z_seuil = 3 (paramétrable)
```

Niveau	Condition sur z	Condition sur le score
(aucune anomalie)	z < 3	score < 0,50
`MEDIUM`	3 ≤ z < 4	0,50 ≤ score < 0,667
`HIGH`	4 ≤ z < 5	0,667 ≤ score < 0,833
`CRITICAL`	z ≥ 5	score ≥ 0,833

Seuils proposés : l'équipe peut les ajuster après les tests réels, à condition de les garder dans la classe de configuration et de les documenter.

5.4 Règles de déclenchement

Anomalie	Série	Condition de déclenchement (toutes à vérifier)
`FREQUENCY_SPIKE`	M1	`z >= z_seuil` ET nombre de logs de la fenêtre ≥ `min_count` (défaut 5) ET au moins `min_baseline` fenêtres d'historique (défaut 10)
`HIGH_SEVERITY_CONCENTRATION`	M2	`z >= z_seuil` sur la proportion `r` ET logs de sévérité élevée ≥ `min_high` (défaut 3) ET logs totaux ≥ `min_count` ET au moins `min_baseline` fenêtres non vides d'historique
`VOLUME_DROP` (optionnel)	M1	`z <= −z_seuil` ET `μ ≥ 5` (sinon une baisse n'a pas de sens)

Démarrage à froid : tant qu'il y a moins de `min_baseline` fenêtres d'historique, aucune anomalie n'est produite pour ce hostname.

5.5 Exemples chiffrés (à reprendre comme tests)

Cas	Historique M1 (N = 10)	Fenêtre cible	Calcul	Résultat attendu
A	8, 10, 9, 11, 10, 12, 9, 10, 11, 10	25 logs	μ = 10 ; s = 1,155 ; z = 15 / 1,155 = 12,99	`FREQUENCY_SPIKE`, score 1,0, `CRITICAL`
B	idem	14 logs	z = 4 / 1,155 = 3,46 ; score = 0,577	`FREQUENCY_SPIKE`, `MEDIUM`
C	idem	12 logs	z = 2 / 1,155 = 1,73	aucune anomalie (z < 3)
D	dix fois 10 (écart-type nul)	11 logs	s_eff = 1 ; z = 1	aucune anomalie (grâce à `s_min`)
E	dix fois 10	40 logs	s_eff = 1 ; z = 30 ; score = 1	`FREQUENCY_SPIKE`, `CRITICAL`
F	5 fenêtres seulement	100 logs	historique < 10	aucune anomalie (démarrage à froid)

Vérification de calcul de l'écart-type, cas A : somme des carrés des écarts = 12 ; 12 / (10 − 1) = 1,333 ; racine = 1,155. Jeu de test de base : pour [2, 4, 4, 4, 5, 5, 7, 9], moyenne = 5, écart-type d'échantillon = 2,138 (écart-type de population = 2).

5.6 Éviter les faux positifs

- Respecter les seuils minimaux (`min_count`, `min_high`, `min_baseline`) : une machine presque silencieuse produit des variations relatives énormes sans signification.
- **Pas de doublon** : l'identifiant d'une anomalie est déterministe, par exemple `type|hostname|début de fenêtre`. Ré-analyser la même fenêtre met à jour le même document.
- Amélioration conseillée : exclure de la baseline les fenêtres déjà signalées comme anomalies, pour qu'un incident prolongé ne « normalise » la référence.

5.7 Pistes d'amélioration (si le temps le permet)

- Méthode robuste (EF12) : score modifié `0,6745 × (x − médiane) / MAD`, seuil usuel 3,5.
- Baseline saisonnière : comparer à la même heure des jours précédents.
- Anomalies sur `username` ou sur les échecs d'authentification : nécessite d'abord d'extraire `username` dans le normaliseur.

6. Modèle de données

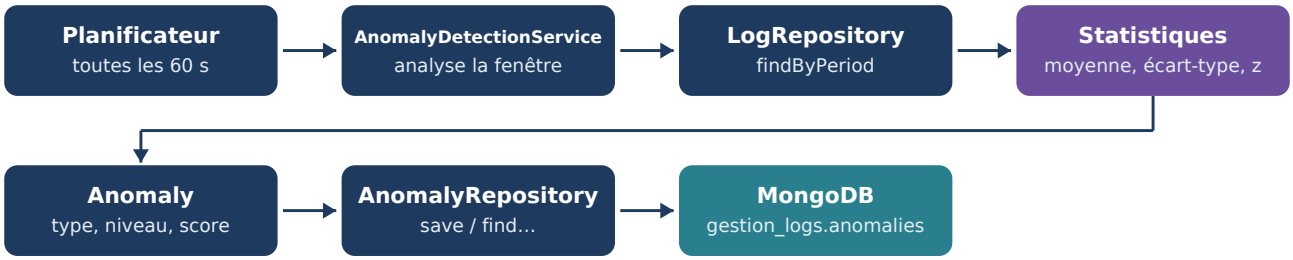
Nouvelles énumérations (package `ma.gestionlogs.model`) : `AnomalyType` (`FREQUENCY_SPIKE`, `HIGH_SEVERITY_CONCENTRATION`, optionnel `VOLUME_DROP`) et `AnomalyLevel` (`MEDIUM`, `HIGH`, `CRITICAL`). **Nouvelle classe** `Anomaly` :

Attribut	Type	Description
`id`	String	Identifiant déterministe `type|hostname|windowStart`
`type`	AnomalyType	Nature de l'anomalie
`level`	AnomalyLevel	Niveau selon le tableau 5.3
`hostname`	String	Machine concernée, ou `ALL` pour le global
`detectedAt`	Instant	Instant de la détection
`windowStart`, `windowEnd`	Instant	Fenêtre analysée
`observedValue`	double	Valeur mesurée (volume ou proportion)
`baselineMean`, `baselineStdDev`	double	μ et s_eff utilisés
`zScore`, `score`	double	Écart normalisé et score entre 0 et 1
`description`	String	Phrase lisible, ex. « 25 logs en 60 s contre 10 en moyenne »

Stockage MongoDB : collection `anomalies`, `_id` = `id`, les `Instant` convertis en `Date` BSON (comme dans `MongoLogRepository`). Index conseillés : `detectedAt` décroissant, `level`, `hostname`.

```
{ "_id": "FREQUENCY_SPIKE|rhel-andreas|2026-11-06T18:30:00Z",
  "type": "FREQUENCY_SPIKE", "level": "CRITICAL", "hostname": "rhel-andreas",
  "detectedAt": ISODate("2026-11-06T18:31:02Z"),
  "windowStart": ISODate("2026-11-06T18:30:00Z"), "windowEnd": ISODate("2026-11-06T18:31:00Z"),
  "observedValue": 25.0, "baselineMean": 10.0, "baselineStdDev": 1.155,
  "zScore": 12.99, "score": 1.0, "description": "25 logs en 60 s contre 10 en moyenne" }
```

7. Architecture proposée



Package	Classe à créer	Responsabilité
`model`	`Anomaly`, `AnomalyType`, `AnomalyLevel`	Modèle et énumérations (section 6)
`repository`	`AnomalyRepository` (interface)	`save`, `findAll`, `findByLevel`, `findByHostname`, `findByPeriod(from, to)`
`repository`	`MongoAnomalyRepository`	Implémentation MongoDB (`replaceOne` avec upsert pour éviter les doublons)
`repository`	`InMemoryAnomalyRepository`	Pour les tests unitaires, comme `InMemoryLogRepository`
`service`	`AnomalyDetectionConfig`	Tous les paramètres (tableau ci-dessous), valeurs par défaut
`service`	`StatisticsUtils`	Fonctions pures : moyenne, écart-type, médiane, MAD, z-score, score
`service`	`AnomalyDetectionService`	`List<Anomaly> analyze(Instant now)` : orchestre l'analyse d'une fenêtre
`service`	`AnomalyScheduler`	Exécute `analyze` toutes les 60 s dans un thread séparé (`ScheduledExecutorService`)

Package	Classe à créer	Responsabilité
racine	`AnomalyDetectionMain`	Lance l'analyse seule, sur les logs déjà en base, sans le récepteur

Déroulement de `analyze(now)` : calculer la fenêtre cible → lire les logs de la période `[début de la baseline ; fin de la fenêtre cible[` avec `findByPeriod` → regrouper par hostname et par fenêtre (fenêtres vides à 0) → calculer μ, s, z pour M1 et M2 → créer les `Anomaly` qui dépassent le seuil → les enregistrer → afficher une ligne de log en console pour chacune.

Paramètre	Défaut	Paramètre	Défaut
Durée de fenêtre `W`	60 s	Seuil `z_seuil`	3,0
Fenêtres de baseline `N`	60	`z_max` (score = 1)	6,0
`min_baseline`	10	`s_min` M1 / M2	1,0 / 0,05
`min_count`	5	Période d'analyse	60 s
`min_high`	3	Sévérités élevées	EMERGENCY, ALERT, CRITICAL, ERROR

> **À savoir**
> **Points à valider avec l'administrateur** : (1) le démarrage du planificateur dans `SyslogTcpReceiver.main` (quelques lignes à ajouter, sans toucher au reste) ; (2) l'ajout de `MongoConnection.getAnomaliesCollection()` ; (3) le choix des tests JUnit (voir section 9). Version 1 : `findByPeriod` + regroupement en Java suffit pour le volume d'un projet de TP ; une agrégation MongoDB (`$group`) pourra venir ensuite.

8. Exigences non fonctionnelles

- **Isolation** : l'analyse ne doit jamais bloquer ni faire échouer la réception ; toute exception est capturée et affichée, le planificateur continue.
- **Tolérance aux pannes MongoDB** : même approche que `MongoLogRepository.save` (message d'erreur, pas de plantage).
- **Idempotence** : relancer l'analyse sur la même période ne crée pas de doublon.
- **Thread-safety** : pas d'état partagé modifiable sans protection.
- **Style** : Java 25, packages `ma.gestionlogs.*`, noms en anglais comme le code existant, commentaires utiles.
- **Performance (objectif)** : analyse d'une période de 61 fenêtres sur 100 000 logs en moins de 5 secondes sur une machine de développement.
- **Évolutivité** : les champs de `Anomaly` et les méthodes de `AnomalyRepository` serviront telles quelles à l'API REST (liste, filtre par niveau, hostname, période) : ne les renommez pas sans prévenir.

9. Tests et critères d'acceptation

Le projet n'a pas encore de JUnit. Deux options : ajouter JUnit 5 (modification du `pom.xml` à faire valider) avec tests dans `src/test/java`, ou, à défaut, des classes de test avec `main` comme `LogNormalizerTest`. Dans les deux cas, chaque scénario ci-dessous doit être automatisé.

#	Scénario	Résultat attendu
T1	`StatisticsUtils` sur [2, 4, 4, 4, 5, 5, 7, 9]	moyenne 5 ; écart-type d'échantillon 2,138
T2	Cas A à F de la section 5.5	Résultats du tableau
T3	Historique avec fenêtres vides	Les trous comptent pour 0 dans M1

#	Scénario	Résultat attendu
T4	Concentration d'erreurs : 3 % d'`ERROR` en moyenne, puis 60 % avec au moins 3 erreurs	`HIGH_SEVERITY_CONCENTRATION` produit
T5	Fenêtre cible avec moins de `min_count` logs	Aucune anomalie
T6	Analyser deux fois la même fenêtre	Un seul document dans `anomalies`
T7	`MongoAnomalyRepository` : sauvegarde puis relecture (base `gestion_logs_test`)	Tous les champs restitués, Instants corrects
T8	MongoDB arrêté pendant `analyze`	Aucun plantage, message d'erreur, le planificateur continue

Test réel avec RHEL

- Laissez la chaîne tourner assez longtemps pour constituer une baseline (au moins `min_baseline` fenêtres, soit 10 minutes avec les valeurs par défaut). Pour une démonstration, réduisez `W` à 10 s via la configuration.
- Depuis RHEL, provoquez une rafale :

```
for i in $(seq 1 300); do logger -p user.err "TEST ERROR burst $i"; done
```

- Attendu : au moins une anomalie `FREQUENCY_SPIKE` et une anomalie `HIGH_SEVERITY_CONCENTRATION` pour la fenêtre concernée, visibles avec :

```
mongosh
use gestion_logs
db.anomalies.find().sort({detectedAt:-1}).limit(5)
```

10. Livrables et définition de « terminé »

Livrable	Contenu
Code	Classes de la section 7 sur la branche `feature/anomaly-detection`, compilant avec `clean package` (`BUILD SUCCESS`)
Tests	T1 à T8 automatisés et réussis ; test réel RHEL documenté (capture de `db.anomalies.find()`)
Documentation	1 à 2 pages pour le rapport : méthode, formules, paramètres retenus, limites, résultats
Démonstration	Rafale de logs depuis RHEL → anomalie visible en base, expliquée par la valeur de z

Ordre de travail conseillé : (1) `StatisticsUtils` et ses tests ; (2) `Anomaly` + `AnomalyRepository` (mémoire puis MongoDB) ; (3) `AnomalyDetectionService` pour M1 ; (4) M2 ; (5) planificateur et exécution manuelle ; (6) test réel RHEL et documentation. Faites une Pull Request vers `integration-logs` à la fin de chaque étape validée.

11. Points d'attention connus

- Les premiers logs RHEL avaient `hostname = 192` ; corrigez le nom de la VM (voir le guide d'installation) pour que le regroupement par machine ait du sens.
- `LogNormalizer` suppose l'année courante pour le RFC 3164 : c'est une raison supplémentaire de travailler avec `receivedTimestamp`.
- MongoDB est local et sans authentification : ne pas l'exposer sur le réseau.
- `InMemoryLogRepository` ne sert qu'aux tests ; la détection lit le stockage via l'interface `LogRepository`.